

## 1. Dataset y preparación

Se usó **California Housing Prices** vía `sklearn.datasets.fetch_california_housing`: 20,640 observaciones, 8 features numéricas continuas (**MedInc**, **HouseAge**, **AveRooms**, **AveBedrms**, **Population**, **AveOccup**, **Latitude**, **Longitude**) y target **median\_house\_value**. Sin nulos ni duplicados. Se detectaron outliers reales (no errores) en **AveBedrms**, **Population**, **AveOccup** y **MedInc** vía regla  $1.5 \times \text{IQR}$ , y el target está *censurado*: 4.68 % de las observaciones topeán en el valor máximo (\$500,001). No se eliminaron (son válidos y reducirían la muestra); se mitigó su efecto escalando features. No hay variables categóricas (la versión sklearn no incluye **ocean\_proximity**), por lo que no se requirió codificación. Se escalaron las 8 features con **StandardScaler** (ajustado solo sobre train) por las diferencias de escala entre variables (p.ej. **Population** en miles vs **Latitude** en decenas). División 70/15/15 (train/val/test): test separado primero y usado una única vez al final.

## 2. Capas y optimizadores de PyTorch (resumen)

**nn.Linear(in,out)**: transformación afín  $y = xW^T + b$ , bloque básico fully-connected. **Activaciones**: **ReLU** ( $\max(0, x)$ , barata pero sufre *dying ReLU*); **LeakyReLU** (permite gradiente pequeño para  $x < 0$ , mitiga lo anterior); **Tanh** (acotada en  $(-1, 1)$ , satura en redes profundas). **nn.Dropout(p)**: apaga aleatoriamente una fracción  $p$  de neuronas en entrenamiento (regularización); se desactiva en `eval()`. **nn.BatchNorm1d**: normaliza activaciones por mini-batch (media 0, var 1) + transformación afín aprendible; estabiliza y acelera el entrenamiento. **Pérdidas**: **MSELoss** (sensible a outliers, penaliza cuadráticamente), **L1Loss** (robusta a outliers, gradiente constante), **SmoothL1Loss** (híbrida, MSE cerca de 0 y L1 lejos).

**Optimizadores**: todos comparten **lr** (tamaño de paso; muy alto diverge, muy bajo estanca) y **weight\_decay** (L2, penaliza pesos grandes). **SGD** actualiza en dirección del gradiente del mini-batch, opcionalmente con **momentum**; es el más simple pero requiere más ajuste manual. **Adam** combina momentum con tasa adaptativa por parámetro (media móvil de gradiente y de su cuadrado); robusto y rápido con poco ajuste, es el más usado por defecto. **RMSprop** adapta el **lr** dividiendo por la raíz de una media móvil del gradiente al cuadrado; predecesor directo de Adam, útil en problemas no estacionarios.

## 3. Resultados: 14 iteraciones (ordenadas por MSE de validación)

Baseline: [64,32], ReLU, Adam (lr=0.001), batch=32, epochs=40, sin regularización. Búsqueda tipo grid dirigido: cada iteración cambia 1–2 variables respecto al baseline.

| #  | Cambio                               | Arq.        | Activ.     | Optim/LR      | Batch/Ep. | MSE           | MAE    | RMSE   |
|----|--------------------------------------|-------------|------------|---------------|-----------|---------------|--------|--------|
| 10 | Batch pequeño                        | [64,32]     | relu       | adam/0.001    | 8/40      | <b>0.2680</b> | 0.3441 | 0.5177 |
| 9  | Batch grande + más epochs            | [64,32]     | relu       | adam/0.001    | 128/80    | 0.2696        | 0.3509 | 0.5193 |
| 8  | LR alto (Adam)                       | [64,32]     | relu       | adam/0.01     | 32/40     | 0.2792        | 0.3585 | 0.5284 |
| 13 | Regularización L1                    | [64,32]     | relu       | adam/0.001    | 32/40     | 0.2797        | 0.3513 | 0.5289 |
| 2  | Arq. más profunda                    | [128,64,32] | relu       | adam/0.001    | 32/40     | 0.2822        | 0.3507 | 0.5312 |
| 6  | SGD + momentum                       | [64,32]     | relu       | sgd/0.01      | 32/40     | 0.2826        | 0.3578 | 0.5316 |
| 4  | Activación LeakyReLU                 | [64,32]     | leaky_relu | adam/0.001    | 32/40     | 0.2832        | 0.3548 | 0.5321 |
| 7  | Optimizador RMSprop                  | [64,32]     | relu       | rmsprop/0.001 | 32/40     | 0.2834        | 0.3636 | 0.5324 |
| 5  | Activación Tanh                      | [64,32]     | tanh       | adam/0.001    | 32/40     | 0.2884        | 0.3604 | 0.5371 |
| 1  | Baseline                             | [64,32]     | relu       | adam/0.001    | 32/40     | 0.2893        | 0.3563 | 0.5379 |
| 12 | Regularización L2                    | [64,32]     | relu       | adam/0.001    | 32/40     | 0.2943        | 0.3624 | 0.5425 |
| 3  | Arq. pequeña (capacidad limitada)    | [16]        | relu       | adam/0.001    | 32/40     | 0.3186        | 0.3887 | 0.5644 |
| 11 | Dropout + BatchNorm                  | [64,32]     | relu       | adam/0.001    | 32/40     | 0.3625        | 0.4074 | 0.6021 |
| 14 | Combo: profunda+LeakyReLU+Dropout+L2 | [128,64,32] | leaky_relu | adam/0.001    | 32/60     | 0.4251        | 0.4214 | 0.6520 |

**Modelo final (#10) sobre test (evaluación única)**: MSE = 0.2761, MAE = 0.3523, RMSE = 0.5254 – consistente con validación, sin señales de que la selección haya sobreajustado a val.

## 4. Análisis de curvas de pérdida

**#1 (Baseline)**: train y val bajan juntos y cierran cerca (train=0.266, val=0.289); leve subida de val tras el epoch 36, señal temprana y suave de overfitting. **#3 (arq. pequeña)**: train y val se estancan juntos en  $\sim 0.32$ , gap $\approx 0$ : *underfitting* claro, el modelo carece de capacidad. **#10 (mejor)**: train (medido en mini-batches ruidosos de tamaño 8) termina por encima del val (0.305 vs 0.268); es un artefacto de medición, no mal ajuste – el modelo generaliza mejor que todos los demás. **#14 (regularización fuerte)**: mejor val en epoch 31 (0.294) pero se sigue entrenando hasta el epoch 59 y val sube a 0.425 mientras train baja; *overfitting tardío* pese a la regularización – *early stopping* habría sido más efectivo que agregar más regularización.

## 5. Discusión

**Mayor impacto positivo/negativo**: el cambio de **batch\_size** (chico, #10, o grande+más epochs, #9) dio la mayor mejora sobre el baseline. El mayor impacto negativo fue combinar dropout+BatchNorm (#11) y sobre todo el combo con L2 y más epochs (#14):

el modelo pasó de generalizar bien a underfitting/overfitting tardío.

**Overfitting/underfitting:** sí, ver #3 (underfitting por poca capacidad) y #14 (overfitting tardío por exceder epochs necesarios). Se identificó comparando las curvas train vs val por epoch; la mitigación recomendada es *early stopping* más que agregar regularización. **¿Ayudó la regularización?** No, en ningún caso mejoró sobre el baseline. L1 (#13,  $\lambda = 10^{-4}$ ) fue casi neutro por ser muy débil; L2 (#12) y dropout+BatchNorm (#11, #14) empeoraron las métricas. Esto ocurre porque el baseline ya generaliza razonablemente bien (gap train/val pequeño), así que añadir restricciones de capacidad solo introduce underfitting en vez de corregir un overfitting que apenas existía.

**Batch size y epochs:** batch chico (8) genera más actualizaciones por epoch y una pérdida de entrenamiento más ruidosa, pero terminó generalizando mejor; batch grande (128) da un entrenamiento más estable/suave pero necesitó el doble de epochs (80) para acercarse al mismo desempeño, ya que hay menos actualizaciones por epoch.

**MSE vs MAE vs RMSE:** MSE/RMSE penalizan más los errores grandes (sensibles a outliers/target censurado detectado en el EDA); MAE da la magnitud típica del error de forma robusta. En el modelo final RMSE (0.525) > MAE (0.352), lo que indica presencia de errores puntuales más grandes que el promedio – coherente con la censura del target en \$500,001.

**Modelo de producción:** arquitectura [64,32] con Adam (lr=0.001), sin dropout/L2 agresivos (no ayudaron aquí), usando *early stopping* sobre val en vez de regularización fuerte, y batch pequeño u optando por más epochs con batch grande. Para seguir optimizando usaría **búsqueda bayesiana** (p.ej. Optuna), más eficiente que grid/random para explorar simultáneamente batch, lr y arquitectura con un presupuesto de iteraciones limitado.

## 6. Conclusiones

El MLP con arquitectura moderada ([64,32]) y Adam ya generaliza bien en este dataset sin necesitar regularización explícita; el principal margen de mejora vino de ajustar batch size/epochs, no de agregar restricciones de capacidad. El target censurado y los outliers reales (no removidos) explican por qué RMSE > MAE consistentemente. El proceso respetó la separación estricta train/val/test, evaluando el conjunto de test una única vez sobre el modelo ya seleccionado.